



FACULTÉ
DES SCIENCES
*Unité de formation
et de recherche*
DÉPARTEMENT
INFORMATIQUE

Université d'Angers

Master 2 Informatique - Conception et Développement

Projet de Fin d'Année

Conception d'un site web pour la gestion des personnels et matériels

Présenté par :

Firmin CAMUS, Antonin CHERRE,
Hugo MAGRET, Clément OUSSET

Encadrants :

M. Jean-Michel RICHER
M. Vincent BARICHARD

Année universitaire 2025 - 2026

Table des matières

1	Introduction	2
1.1	Contexte et Objectifs	2
1.2	Enjeux et Périmètre	3
2	Visualisation et Interface Utilisateur	4
2.1	Représentation Interactive des Plans	4
2.2	Export en PDF	4
2.3	Capture d'écran de l'Interface	5
2.3.1	Vue 3D et rendu procédural	5
2.3.2	Import JSON des étages	5
3	Architecture et Choix Techniques	6
3.1	Frontend : Angular	6
3.1.1	Rendu 3D et interactivité procédurale	6
3.2	Backend : Node.js	7
3.2.1	Prisma ORM	7
3.3	Base de données : PostgreSQL	7
3.4	Schéma de fonctionnement	7
3.5	Déploiement et Conteneurisation : Docker	8
3.6	Collaboration : GitHub	8
3.7	Organisation du Projet	8
4	Conception de la Base de Données	9
4.1	Modèle Logique de Données (MLD)	9
4.2	Détail des tables et contraintes	10
5	Conception Logicielle	11
5.1	Schéma d'architecture applicative	11
5.2	Principes de développement	12
5.2.1	Services métiers complémentaires	12
5.2.2	Authentification JWT et intercepteur Angular	12
5.2.3	Gestion d'état et réactivité (RxJS)	12
5.3	Sécurité et gestion des mots de passe	13
6	Annexes	14
6.1	Planification du projet	14

Chapitre 1

Introduction

Dans le cadre de notre dernière année de Master 2 en Informatique, parcours Conception et Développement, il nous a été confié la réalisation d'un projet de fin d'études axé sur la gestion du personnel et du matériel du département informatique.

1.1 Contexte et Objectifs

L'objectif principal de ce projet est de concevoir et développer une web app complète permettant de recenser et de gérer les personnels ainsi que le matériel informatique au sein des différentes salles du département.

Ce système doit permettre de classer les espaces selon leur type (bureau, salle de classe, salle de réunion...) et de gérer leur capacité d'accueil. Il est également nécessaire de pouvoir suivre l'équipement présent dans chaque salle (prises réseaux, vidéoprojecteurs, tableaux...), et de gérer dynamiquement les affectations du personnel.

Un point important de ce projet est la gestion des plans de la faculté au format JSON. Ce format structuré est facile à manipuler et à créer manuellement, permettant une maintenance et une évolution simplifiées. Les plans JSON sont ensuite rendus de manière interactive directement dans le navigateur, affichant les zones cliquables et s'adaptant facilement à différents écrans. De plus, l'application offre la capacité d'exporter en PDF les plans qui auront été modifiés par les utilisateurs, afin de conserver un rendu imprimable et partageable.

1.2 Enjeux et Périmètre

Au-delà du simple développement d'un système d'information de type CRUD (Create, Read, Update, Delete), les attentes de ce projet portent sur la maintenabilité de l'application et son ergonomie. Parmi les fonctionnalités majeures attendues, nous devons implémenter :

- **Plans interactifs 2D et 3D** : Le système doit permettre de visualiser les locaux de manière interactive via un plan 2D exploitant le format JSON pour plus de flexibilité. Un plan 3D complète cette vue, offrant une perspective supplémentaire des espaces. En cliquant sur une salle, un panneau latéral affiche en temps réel les détails de la salle (nom, capacité, type, étage), la liste complète du matériel présent et l'affectation du personnel.
- **Gestion complète de l'inventaire** : L'application doit permettre de consulter, modifier et gérer l'ensemble des données : matériel, personnel, salles, types d'équipements. Les utilisateurs peuvent ajouter, supprimer ou éditer des éléments. Le matériel peut être déplacé entre les salles directement via l'interface.
- **Authentification et profils utilisateurs** : Un système de connexion/déconnexion sécurisé. Les administrateurs disposent d'une interface dédiée pour gérer la liste du personnel et les droits d'accès.
- **Export en PDF** : Capacité à exporter les plans et états des étages (avec matériel et personnels affectés) au format PDF pour archivage et partage.

Ce rapport détaillera l'ensemble du processus de création de cette plateforme, depuis nos choix technologiques orientés "industrie" jusqu'à la mise en place de notre architecture logicielle.

Chapitre 2

Visualisation et Interface Utilisateur

L'interface utilisateur est au cœur de l'expérience de gestion des locaux. Elle doit être intuitive, réactive et permettre une visualisation claire et interactive des plans de la faculté.

2.1 Représentation Interactive des Plans

L'application utilise des plans au format JSON chargés depuis la base de données. Ces plans sont ensuite rendus interactivement dans le navigateur. Ce format structuré offre plusieurs avantages :

- **Interactivité** : Les zones correspondant aux salles peuvent être cliquables et affichent en temps réel les informations détaillées (capacité, personnel, équipement).
- **Flexibilité** : Le format JSON est facilement manipulable et peut être créé ou modifié manuellement, permettant une maintenance simplifiée des plans.
- **Modifiabilité** : Les utilisateurs peuvent effectuer des modifications directement sur le plan (ajout / retrait de matériel et personnel).

2.2 Export en PDF

Une fonctionnalité clé de l'application est la possibilité d'exporter les plans modifiés au format PDF. Cela permet :

- De conserver un enregistrement imprimable des modifications apportées.
- De partager facilement les plans avec des parties externes.
- D'archiver l'état des locaux à une date donnée pour traçabilité et historique.

2.3 Capture d'écran de l'Interface

La figure ci-dessous illustre l'interface principale de l'application, montrant la visualisation du plan JSON rendu interactivement.

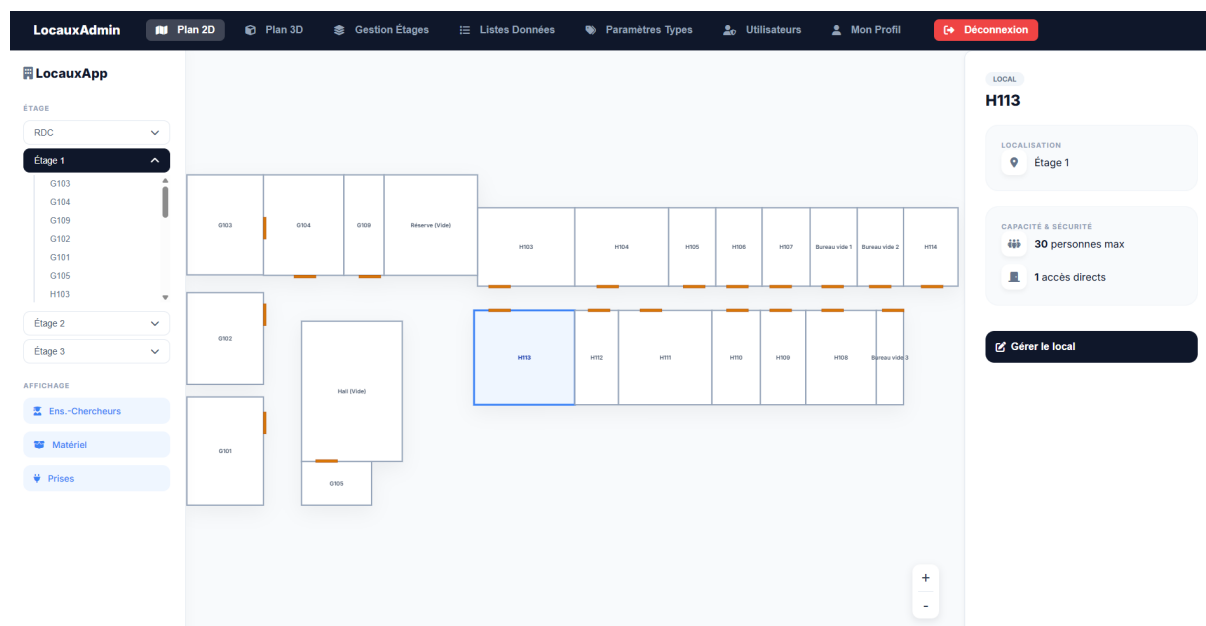


FIGURE 2.1 – Interface de visualisation des plans JSON avec options d'export PDF

2.3.1 Vue 3D et rendu procédural

L'application propose également une vue 3D (Three.js) générée de façon procédurale à partir des coordonnées 2D contenues dans les plans JSON. Les murs et volumes sont construits à l'aide de primitives (par exemple **BoxGeometry**) et texturés de manière simple pour conserver une lisibilité optimale. La sélection d'éléments en 3D repose sur le ray-casting (projection d'un vecteur depuis la caméra vers le curseur) et la navigation utilise **OrbitControls** pour rotation et zoom.

2.3.2 Import JSON des étages

Les plans sont importés au format JSON via une API dédiée. Le format attendu contient les métadonnées de l'étage et des listes structurées : `rooms[]`, `doors[]`, `equipments[]`, `sockets[]`. L'import côté backend est transactionnel : lors d'un import massif nous utilisons `prisma.$transaction()` pour garantir que l'opération est atomique (soit tout est appliqué, soit rien en cas d'erreur). Le backend valide également le format et normalise les types (création automatique des `room_type` ou `equipment_type` si nécessaire).

Chapitre 3

Architecture et Choix Techniques

Au-delà de répondre aux exigences de maintenabilité et de simplicité de déploiement imposées par le sujet, ce projet de fin d'études porte sur un certain objectif : s'entraîner à développer selon les pratiques et standards de l'industrie. Il aurait été possible de réaliser cette application intégralement en Python, mais nous avons plutôt opté pour une stack technologique composée d'outils professionnels largement utilisés dans le monde de l'entreprise, afin d'acquérir une expertise transférable et valorisable dans notre future carrière.

Nos choix technologiques ont ainsi été guidés par deux critères majeurs : la robustesse de la solution et notre volonté de consolider des compétences demandées dans le monde de l'entreprise. Cette approche garantit à la fois une application fiable et maintenable, mais constitue aussi un investissement pédagogique majeur pour notre insertion professionnelle.

3.1 Frontend : Angular

Pour l'interface utilisateur, notre choix s'est porté sur le framework Angular. Ce choix s'explique tout d'abord par une compétence déjà présente au sein de l'équipe (deux membres avaient déjà utilisé cette technologie). De plus, Angular est aujourd'hui un standard de l'industrie pour les applications web complexes (Single Page Applications). Sa structure basée sur les composants et l'utilisation de TypeScript garantissent un code typé, maintenable et évolutif.

3.1.1 Rendu 3D et interactivité procédurale

Pour dépasser les limites des représentations planes, nous avons intégré la bibliothèque Three.js. Ce moteur permet une génération procédurale des volumes 3D à partir des coordonnées JSON 2D. Chaque mur est calculé et généré dynamiquement, offrant une immersion architecturale. L'interactivité est assurée par deux mécanismes complémentaires : les OrbitControls pour une navigation fluide (rotation, zoom) autour du modèle, et une technique de Raycasting qui projette un vecteur depuis la caméra vers le curseur pour intercepter les surfaces cliquables, permettant ainsi une sélection précise des salles dans l'espace tridimensionnel.

3.2 Backend : Node.js

Bien que notre formation de 5 ans en informatique nous ait permis d'acquérir une grande expertise sur le langage comme Java, nous avons décidé d'implémenter notre API Backend en Node.js. Ce choix est doublement stratégique. D'une part, Node.js est extrêmement populaire dans l'industrie actuelle pour le développement de micro-services et d'API REST. D'autre part, cela nous permettait de sortir de notre zone de confort académique (Java) pour relever un défi technique pertinent pour notre future insertion professionnelle.

3.2.1 Prisma ORM

Le backend utilise Prisma comme ORM pour interagir avec PostgreSQL. Prisma fournit un client typé généré à partir du schéma de données, facilite la rédaction de requêtes et la gestion des migrations. Nous utilisons également `prisma.$transaction()` pour réaliser des opérations atomiques (par exemple l'import transactionnel des plans d'étage), et un singleton de connexion (`back/config/prisma.js`) pour centraliser l'initialisation du client Prisma et éviter les dépassements de connexions en environnement serveur.

3.3 Base de données : PostgreSQL

Le système devant gérer de potentielles écritures simultanées (modification de salles, ajout de matériels), nous avons rapidement écarté SQLite. Très léger, SQLite nous semblait trop faible pour nos besoins. Notre choix final s'est donc porté sur PostgreSQL, un Système de Gestion de Base de Données Relationnelle (SGBDR) robuste et open-source. Il s'intègre également de manière fluide avec Node.js.

3.4 Schéma de fonctionnement

Le schéma suivant illustre le workflow principal de l'application : l'utilisateur interagit avec le frontend Angular, qui transmet des requêtes HTTP au backend Node.js. Ce backend traite la logique métier, interroge PostgreSQL pour lire ou modifier les données, puis renvoie les réponses au frontend.

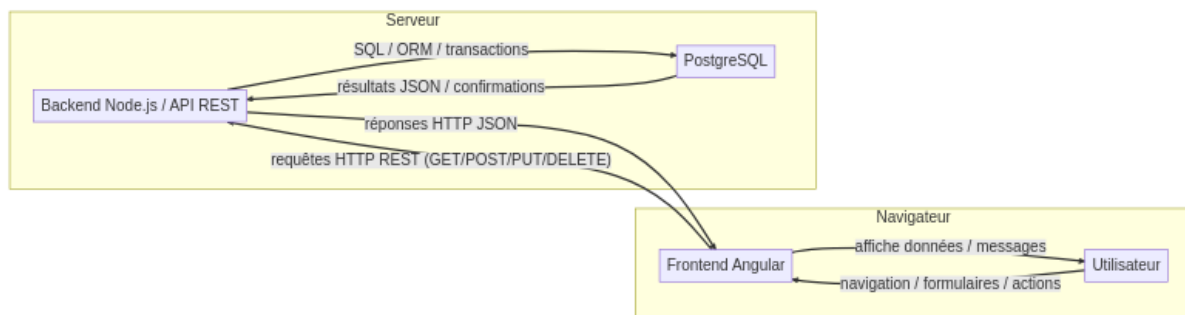


FIGURE 3.1 – Workflow de communication entre l'utilisateur, le frontend, le backend et la base de données

3.5 Déploiement et Conteneurisation : Docker

Afin de garantir que notre site soit "facilement installable" sur n'importe quel serveur du département informatique, nous avons intégré Docker dès le premier jour de développement. Docker nous permet d'encapsuler notre base de données, notre backend et notre frontend dans des conteneurs isolés. L'utilisation de `docker-compose` permet de lancer l'intégralité de l'environnement avec une seule commande, résolvant ainsi les problèmes de compatibilité entre les systèmes d'exploitation, que ce soit pour nous développeurs du projet, ou pour les futurs utilisateurs.

3.6 Collaboration : GitHub

Travaillant en équipe de quatre, nous avons choisi GitHub comme plateforme de gestion des versions du code. Cette approche nous a permis de maintenir une traçabilité rigoureuse des modifications, de créer des releases versionnées pour nos livrables, et de répartir le travail via des tickets (issues). Concrètement, les issues servent à définir et prioriser les tâches : chaque tâche donne lieu à une branche dédiée puis à une pull request (merge request) regroupant les changements et facilitant la revue de code et la validation avant fusion. Ce choix reproduit les bonnes pratiques observées dans les équipes professionnelles et facilite la collaboration entre membres de l'équipe.

3.7 Organisation du Projet

Afin de garantir une séparation stricte des responsabilités (separation of concerns), notre dépôt de code est structuré de la manière suivante :

```
Gestion_locaux/
|-- back/          # API Node.js (Logique metier et acces DB)
|-- front/         # Application Angular (Interface utilisateur)
|-- database/      # Fichiers SQL et scripts d'initialisation PostgreSQL
|-- rapport/       # Sources LaTeX de ce document
+-- docker-compose.yml # Orchestrateur des differents conteneurs
```

Cette arborescence permet à chaque membre de l'équipe de se repérer facilement et facilite le déploiement automatisé.

Chapitre 4

Conception de la Base de Données

La conception de la base de données est au cœur de notre application. Pour garantir l'intégrité des données des salles, du personnel et du matériel, nous avons choisi une modélisation relationnelle sous PostgreSQL. Les tables utilisent des identifiants UUID pour garantir l'unicité des données ; les coordonnées des salles sont stockées au format JSONB afin de conserver une représentation structurée et extensible des positions et dimensions des zones sur les plans.

4.1 Modèle Logique de Données (MLD)

Le schéma ci-dessous présente l'organisation des tables et les relations entre elles. Il met l'accent sur les contraintes d'intégrité référentielle (clés primaires et étrangères), qui assurent la cohérence des informations de l'inventaire et de l'occupation des salles.

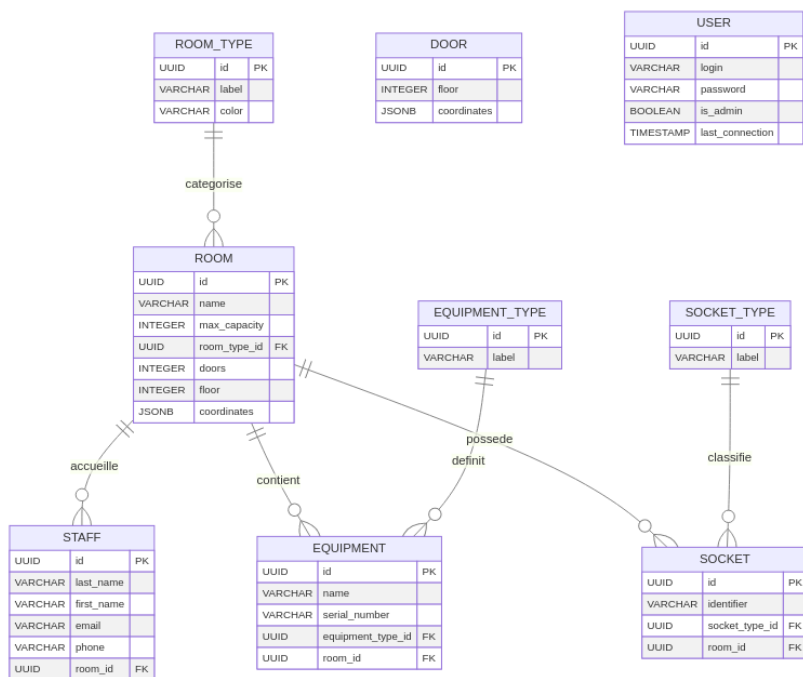


FIGURE 4.1 – Schéma relationnel de la base de données PostgreSQL

4.2 Détail des tables et contraintes

- **Table room** : Elle modélise les espaces physiques. Chaque salle est caractérisée par sa capacité maximale, son type, son étage et ses coordonnées pour le plan.
- **Table staff** : Elle contient le personnel affecté aux locaux. La relation `room_id` permet de savoir où chaque employé est positionné.
- **Table user** : Cette table stocke les comptes d'accès à l'application, ainsi que les informations de profil. Le champ `is_admin` distingue les utilisateurs avec droits d'administration des utilisateurs standards.
- **Tables de référence** : `room_type`, `equipment_type` et `socket_type` garantissent la cohérence des catégories utilisées dans les salles, le matériel et les prises.
- **Table room_type** : contient le label et un champ `color` (couleur hexadécimale) utilisé par le frontend pour coloriser les salles dans les listes.
- **Table equipment** : Elle représente l'inventaire matériel d'une salle. Chaque équipement est lié à un type et peut être rattaché à une salle.
- **Table socket** : Elle répertorie les prises présentes dans chaque salle et les associe à un type de connectique.
- **Table door** : Elle modélise les points d'accès (portes) sur les plans, avec leur étage et leurs coordonnées stockées en `JSONB`.

Le schéma présenté regroupe donc les tables principales suivantes : `room`, `door`, `staff`, `equipment`, `socket`, ainsi que les tables de référence `room_type`, `equipment_type`, `socket_type` et la table `user` pour la gestion des accès.

La modélisation adopte des règles solides : les types de référence sont supprimés en cascade, tandis que les dépendances sur les salles sont gérées avec `SET NULL` lorsque cela est pertinent, afin de préserver les données historiques et limiter les pertes involontaires.

Chapitre 5

Conception Logicielle

Après avoir défini la persistance, nous avons structuré l'application en deux couches principales : le backend Node.js, qui fournit l'API REST, et le frontend Angular, qui orchestre l'interface et la manipulation des données côté client.

5.1 Schéma d'architecture applicative

Le schéma ci-dessous illustre l'architecture applicative de l'application Angular et les principaux services métiers. Il montre comment les composants, les modèles et les services collaborent pour gérer les locaux, le personnel, le matériel et les prises. Bien que présenté sous forme de diagramme synthétique, il ne s'agit pas d'un diagramme de classes UML strict mais d'une représentation opérationnelle des composants et de leurs interactions.

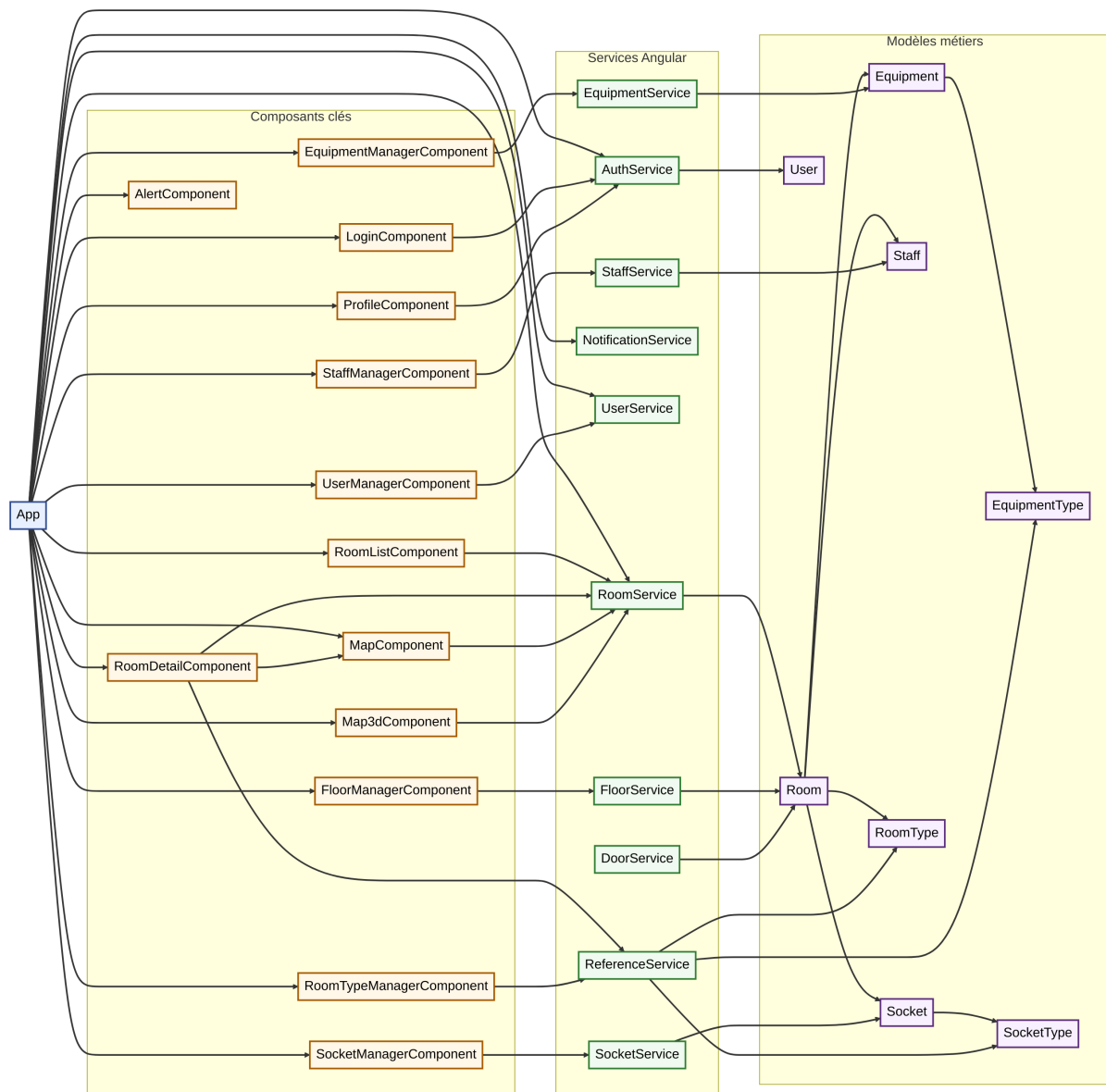


FIGURE 5.1 – Schéma d'architecture applicative - Angular

5.2 Principes de développement

La conception logicielle privilégie une séparation claire des responsabilités entre :

- **Modèles** : Les classes Room, Equipment, Staff, RoomType, EquipmentType et SocketType définissent les entités manipulées par le client.
- **Services** : Les services Angular (RoomService, AuthService, ReferenceService, EquipmentService, StaffService, SocketService) encapsulent l'accès aux API et la logique de synchronisation des données.
- **Composants** : Les composants gèrent l'affichage et l'interaction utilisateur : liste des salles, détail d'une salle, gestion des types, connexion, etc.

Cette architecture améliore la maintenabilité du projet. Le backend reste léger et modulaire, tandis qu'Angular assure un rendu dynamique et une expérience utilisateur fluide.

5.2.1 Services métiers complémentaires

FloorService et import transactionnel Le service `FloorService` orchestre la gestion des étages côté client et appelle l'API d'import. Côté backend l'import des plans est effectué dans une transaction (`prisma.$transaction()`) afin d'assurer l'atomicité et d'éviter des états partiels en cas d'erreur.

NotificationService et AlertComponent Le `NotificationService` fournit une API de notifications centralisée (toasts) utilisée par tous les composants pour afficher les succès, erreurs et avertissements. Le composant `AlertComponent` consomme ce service et affiche des messages persistants ou temporaires selon le cas d'utilisation.

5.2.2 Authentification JWT et intercepteur Angular

L'authentification utilise un mécanisme token-based : après connexion, le backend émet un JWT signé qui est renvoyé au client. Le token est stocké côté client (par exemple `localStorage` sous la clé `auth_token`) et un intercepteur HTTP Angular (`auth.interceptor.ts`) ajoute automatiquement l'en-tête `Authorization: Bearer <token>` aux requêtes sortantes. Le backend vérifie le token via un middleware (`verifyToken`) et applique des contrôles supplémentaires (`verifyAdmin`) pour les routes réservées aux administrateurs.

5.2.3 Gestion d'état et réactivité (RxJS)

Au-delà de la structuration classique, nous avons implémenté une gestion d'état réactive basée sur RxJS. L'utilisation de `BehaviorSubject` dans nos services, notamment `RoomService` et `FloorService`, agit comme une source de vérité unique. Lorsqu'une donnée est modifiée par un composant, le flux de données propage instantanément la mise à jour à tous les autres composants abonnés. Ce patron de conception avancé permet une interface fluide sans rechargement de page, garantissant que la vue 2D, 3D et les listes soient toujours cohérentes avec l'état actuel de la base de données.

5.3 Sécurité et gestion des mots de passe

Le backend gère l'authentification des utilisateurs et le stockage sécurisé des mots de passe. Concrètement, les mots de passe sont hachés côté serveur (fonction `hashPassword` dans `back/config/auth.js`) à l'aide d'un dérivé sécurisé (`scrypt`) avant d'être persistés en base. La vérification (`verifyPassword`) compare de façon sécurisée le mot de passe fourni avec le haché stocké. Par mesure de sécurité, il est recommandé de changer le mot de passe administrateur par défaut fourni dans les scripts d'initialisation dès le premier déploiement, et d'utiliser des secrets/variables d'environnement pour les identifiants en production.

Chapitre 6

Annexes

6.1 Planification du projet

La figure ci-dessous présente la planification réelle et itérative du projet.

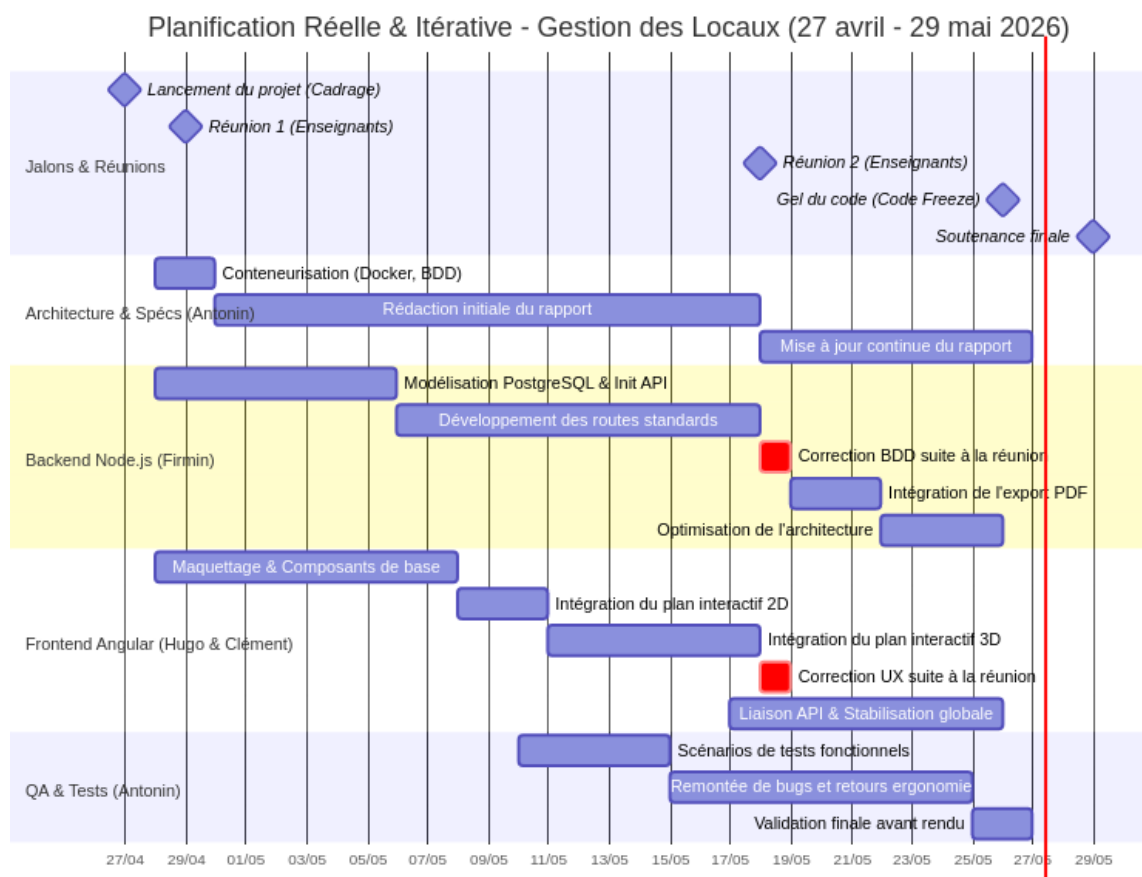


FIGURE 6.1 – Planification du projet — Gantt